



Práctica 7: Pruebas, eficiencia, búsqueda y ordenamiento.

Objetivos

- Aprender a desarrollar programas aplicando criterios de eficiencia en la cantidad de instrucciones ejecutadas y en el manejo de las estructuras de datos.
- Implementar operaciones administrativas básicas como ordenamiento y búsqueda sobre listas.
- Aprender a analizar, comparar, implementar y reutilizar algoritmos de ordenamiento existentes.
- Aplicar los conocimientos adquiridos acerca de prueba de programas.

Introducción

En la universidad de Antioquia se desea generar algunas estadísticas acerca de los estudiantes del primer semestre de Medicina. Para ello, se cuenta con los números de documento de los estudiantes del primer semestre, el código de todos los cursos del primer semestre de dicha carrera y la respectiva nota que cada estudiante ha obtenido en cada uno de los cursos. El punto de partida es la siguiente estructura de datos a partir de la cuál usted debe llevar a cabo su desarrollo:

		cursos				
		C14	...	...	...	D23
estudiantes	10915	3.5	4.4	-1	...	2.1
	...	0.1	...			...
	...	...				...
	14733	4.9	...	...	...	...

Notas del estudiante de documento 10915, en cada uno de los cursos del primer semestre. El -1 expresa que el estudiante canceló el curso correspondiente a esa columna.

Notas del curso C14, para cada uno de los estudiantes del primer semestre. El -1 expresa que el estudiante correspondiente a esa fila canceló el curso.

La estructura está conformada por dos listas y una matriz (lista bidimensional). En el diagrama se puede observar que dichas estructuras están desacopladas, lo cual quiere decir que no están anidadas entre sí. Sin embargo, deben administrarse de forma conjunta o **paralela** dado que los elementos en las listas se encuentran dispuestos de tal manera que guardan una relación posicional con aquellos dentro de la matriz. Esto se puede constatar en las acotaciones presentadas con líneas azules alrededor de las filas y columnas de la matriz. En dicha matriz, se pueden identificar las notas del estudiante i en el curso j . El valor -1 dentro de la matriz indica que el estudiante i canceló el curso j , mientras que el valor -2 indica que el estudiante i no está inscrito en el curso j .

Esta práctica debe resolverse usando la estructura de datos propuesta y considerando sus propias implementaciones de funciones para administrar las listas involucradas. De ser necesario, puede definir variables o estructuras de datos auxiliares para satisfacer los requisitos del programa.

Sección I: Requisitos funcionales

Descripción de las funcionalidades requeridas en el menú principal:

1. **Cargar datos desde archivo:** Permite leer el archivo database.txt y cargar las estructuras de datos del programa.
2. **Eliminar estudiante:** Permite eliminar el estudiante correspondiente a un número de documento dado (y sus notas).
3. **Agregar estudiante:** Permite agregar un estudiante nuevo (identificado por su número de documento) junto con las respectivas notas de todos sus cursos.
4. **Promedio de estudiantes:** Calcula el promedio de todos los estudiantes del curso, devolviendo el resultado en una lista. Este resultado se debe mostrar por pantalla.
5. **Promedio de cursos:** Calcula el promedio de notas en todos los cursos, devolviendo el resultado en una lista. Este resultado se debe mostrar por pantalla.
6. **Tres notas mayores:** Calcula las 3 mayores notas de un curso dado, e imprime los números de documento de dichos estudiantes. Si varios estudiantes coinciden en las notas, deben reportarse todos.
7. **Menor nota de estudiante:** Retorna el código del curso en el cual ha obtenido su menor nota un estudiante dado.
8. **Ordenar promedios estudiantes:** Imprime por pantalla a los estudiantes ordenados según su promedio de notas, de mayor a menor. Para esta funcionalidad usted debe usar obligatoriamente una estrategia de ordenamiento burbuja.
9. **Ordenar estudiantes por cantidad de cursos:** Ordena los estudiantes según la cantidad de materias cursadas e imprime el resultado por la pantalla. Para esta funcionalidad usted debe usar obligatoriamente una estrategia de ordenamiento por selección.
10. **Opcional:** Al salir del programa, el archivo de datos debe ser actualizado. (0.3 puntos, actividad opcional)

Requisitos no funcionales:

- Eficiencia en el diseño de las estructuras de datos y en los procesos. Lea el apartado eficiencia más abajo en este documento.
- El código generado debe hacer uso de módulos y estar debidamente documentado.
- Para la construcción de esta solución, en ningún caso se debe hacer uso de diccionarios.

Sección II - Manejo de archivos

El archivo de de datos, llamado “database.txt” , presenta el siguiente formato:

```
CodigoC1, CodigoC2, ..., CodigoCN
NdocE1 NdocE2 ... NdocEM

NotaE1C1 NotaE1C2 NotaE1CN
NotaE2C1 NotaE2C2 NotaE2CN
...
NotaEMC1 NotaEMC2 NotaEMCN
```

En la primera línea, se tienen los códigos de cada uno de los N cursos, separados por coma. En la segunda, los M documentos de los estudiantes.

Luego se tienen las M líneas, en cada línea están las N notas de cada estudiante.

Sección III - Eficiencia

En está práctica, uno de los elementos fundamentales de la evaluación será la eficiencia.

Documente por escrito, a manera de informe en word, dos funciones en las que haya logrado reducir la cantidad de iteraciones requeridas a través de alguna estrategia analítica (por ejemplo, incrementando la complejidad conceptual). Incluya la versión original y la versión optimizada del código de la función, acompañándola de la explicación del análisis que aplicó para lograr la optimización. Se requiere que la optimización sea significativa, disminuyendo la cantidad de iteraciones o el movimiento de datos al menos en un 5%.

Es importante que considere realizar la menor cantidad de movimientos de datos posibles. Para lo cual sugiere utilizar alguna estrategia de indexación. Es decir, utilización de índices como referencia a las posiciones en listas. Durante la presentación de este documento, el profesor brindará un ejemplo sobre esta estrategia.

Sección IV - Pruebas

Diseñe y documente tres pruebas de caja negra, cada de ellas debe verificar la correctitud de una función distinta entre las implementadas en su solución. Al menos una de estas debe trabajar sobre grandes volúmenes de datos. Para el diseño de estas pruebas, se deben considerar todas las pautas dadas en las clases teóricas y de laboratorio.